

Trabajo Práctico Integrador - Programación I

[Buscador de Países]

Alumnos - Grupo n° 109

Joaquin Ignacio Rodriguez, Guido Toledo Gonzalez.

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Programación I

Docente Titular

Alberto Cortez

Docente Tutor

Juan Sarmiento (C-25) y Sofía Fernández (C-13)

15 de junio de 2025

Tabla de contenido

Introducción y objetivos	3
Marco teórico	
Listas	4
Diccionarios	5
Condicionales	6
Funciones	7
Archivos .CSV	9
Decisiones Técnicas y Arquitectura	10
Diagrama de Flujo y Ejemplos de ejecucion	11
Referencias	15
Dificultados y Conclusiones	16
Links	16

Trabajo Práctico Integrador

Introducción

El proyecto trata sobre crear un programa en python que lea datos sobre países a partir de un archivo CSV, y a usar estos datos para realizar consultas, ofrecer estadísticas relevantes, filtrar, ordenar y agregar datos de forma permanente al archivo CSV.

Objetivos

Que los participantes logren:

- Definir de manera profunda y precisa los conceptos teóricos utilizados.
- Crear un programa funcional y a prueba de errores.
- Realizar una presentación profesional y detallada sobre el programa.

espacio dejado intencionalmente en blanco

Marco Teórico

La siguiente sección definirá los conceptos aplicados con sus correspondientes fuentes bibliográficas.

Listas¹

Una de las estructuras de datos más versátiles de Python es la lista, es utilizada para agrupar valores, separados por comas, entre corchetes []. A cada uno de los valores dentro de la lista les decimos *items*, una lista puede ser pensada como una secuencia de valores y una lista dentro de otra lista se dice que esta anidada²

```
frutas = ["banana", "manzana", "pera"]  
numeros = [1,2,3,4,5]
```

Fig 1. Lista con frutas y numeros

Las listas pueden contener distintos tipos de datos, pero usualmente tienen el mismo tipo de dato:

```
numeros_y_frutas = [1, "banana", "pera", {"anana":5}]
```

Fig 2. Lista con elementos de distinto tipo

Las listas son un objeto mutable, es decir, es posible cambiar su contenido, a diferencia de tipos de datos como strings que son inmutables; en el proyecto las listas fueron utilizadas para almacenar los datos obtenidos del archivo CSV en la variable “países”, almacenando en la lista diccionarios, donde cada diccionario es un elemento, y donde cada valor (*value*) se corresponde a una columna que funciona como su clave(*key*) (ejemplo, “poblacion” : 5000).

Las listas contienen elementos ordenados donde cada posición corresponde a un elemento y se empieza a contar a partir de la posición 0.

```
spam = ["cat", "bat", "rat", "elephant"]  
      ↑   ↑   ↑   ↑  
spam[0] spam[1] spam[2] spam[3]
```

Fig 3. Lista que muestra a que indice pertenece cada item o elemento.³

Las listas son indexadas y puede aplicárseles slicing para acceder a elementos particulares de la lista.

```
frutas[0] #Accede a banana
frutas[-1] #Accede a pera
frutas[:] #Accede a todos los elementos
```

Esto fue utilizado en la función **ordenar_paises()** para hacer display al usuario de los países ordenados por nombre, población o superficie, , la función **estadísticas** se usó [0] para poder inicializar las variables mínimo y máximo con el primer país de la lista.

Se eligió el tipo de dato lista para los países porque permite hacer operaciones de ordenamiento de forma sencilla mediante la función `sorted()`

Diccionarios⁴

A diferencia de las listas, a las cuales se accede a sus elementos mediante índices numéricos, los diccionarios son indexados por llaves (*keys*) que pueden ser cualquier tipo de dato inmutable (como strings), y estas keys son usadas para acceder a los valores, las strings y números siempre pueden ser keys, en el caso del proyecto las keys son strings y corresponde a la primera fila de el archivo CSV, es decir, el espacio ocupado por las columnas (país, población, superficie y continente). ¿Por qué las llaves son inmutables? Porque debido a que usan un hash value calculado a partir de la llave para encontrar el valor, si la llave cambiara también cambiaría el hash value.⁵

Los diccionarios pueden ser pensados como pares de clave : valor , con el requerimiento de que las keys sean únicas (no pueden repetirse en el diccionario). Los distintos pares clave, valor son separados por comas, y el diccionario está entre corchetes `{ }`

```
países = [{"pais": "Chile", "poblacion":18480432, "superficie": 756626, "continente": "america del sur"}]
```

Fig 2.Lista llamada países con un diccionario como elemento en posición 0

Esta estructura de datos posee la ventaja de ser fácilmente legible y accesible, usada en el programa permite hacer cambios fácilmente a los valores de la clave buscada (por ejemplo, es fácil acceder a el valor “poblacion” de un elemento).

Los diccionarios son además una estructura de datos rápida, debido al utilizar *hash tables*, toman tiempo constante $O(1)$ para obtener una llave.⁶

Condicionales

Match statements⁷

Un match statement es una expresión que compara un valor a patrones sucesivos de uno o más bloques case. Hace que el código sea más fácil de leer que usando el condicional if, siendo por este motivo útil para nuestro programa porque puede ser utilizado para opciones de menú, siendo la expresión elegida el input del usuario (ejemplo: opcion 1) y el bloque case a comparar el número de la opción.

```
opcion = input("Ingrese una opcion: ")

match int(opcion):
    case 1:
        pass
        #codigo de opcion 1.
    case 2:
        pass
        #codigo de opcion 2.
    case 3:
        pass
        #codigo de opcion 3.
    case _:
        pass
        #Codigo a ejecutar en caso de que las otras condiciones no se cumplan.
```

Fig 3. Ejemplo de un condicional match

En caso de que la expresión evaluada sea igual al case esto ejecuta el código correspondiente al case, y dejan de evaluarse sucesivos case. En caso de que los otros case no se cumplan, el utilizar "case _" se ejecuta, esto es una ejecución de código por defecto utilizado para casos impredecibles en donde ninguna de las opciones anteriores coincide.

if statements^{8.9}

Un if statement es utilizado para ejecutar un bloque de código si una condición es evaluada como verdadera, un if statement puede tener cero ó varios elif statements que lo siguen, y puede tener cero ó un único else statement, primero es evaluada la condición if, luego las condiciones elif (si las hay) y por último la condición else (si la hay). Estas operaciones booleanas son ejecutadas en orden, cuando una condición es evaluada como true y ejecutada el resto de las partes del if statements dejan de ser evaluadas ó ejecutadas. En caso de que todas las condiciones if, elif sean evaluadas como False la condición else es ejecutada (si la hay).

Los if statements son fundamentales para operaciones booleanas simples, como por ejemplo, chequear si hay países cargados utilizando el código:

```
def estadisticas(países): # Opcion 6
    """Muestra estadísticas generales de los países"""
    if not países:
        print("No hay países cargados.")
        return
```

Fig 4. Función que chequea que haya a elementos en la lista países

En el caso de que haya muchas condiciones a evaluar, como las opciones del menú, por cuestiones de legibilidad es mejor utilizar un condicional match-case.

A veces es útil tener un if statement sin código a ejecutar en caso de que la condición sea verdadera, utilizando el *reserved keyword* pass, esto es usado como placeholder para código que todavía no escribimos, esto fue usado durante la creación de nuestro programa para no olvidar crear ciertos bloques de código.

```
if x < 0:
    pass                # TODO: need to handle negative values!
```

Fig 5. if statement usado como placeholder¹⁰

Funciones

Una función está diseñada para realizar una tarea específica dentro de un programa, permitiendo así la reutilización del código para cumplir el principio de “Don’t repeat yourself”. Permite dividir problemas en partes más pequeñas, y hacer el código más fácil de mantener y reutilizable. En lugar de escribir el mismo código en distintas partes del programa se crea una función y se la invoca para realizar esta tarea.

Una función es creada mediante la *reserved keyword* def, seguido del nombre de la función y de parentesis en los cuales están encapsulados sus parámetros (si los hay). Estos parámetros son valores utilizados por la función durante su ejecución, a los datos pasados a la función se los llama argumentos y pueden ser posicionales ó keyword arguments, en el caso de nuestro programa se utilizaron argumentos posicionales.

```
def print_twice(bruce):  
    print(bruce)  
    print(bruce)
```

Fig 6. Función creada con un argumento posicional¹¹

El primer statement de la función puede ser un string literal creado por comillas triples; a esto se le dice documentation string ó docstring. Estos docstring son buena práctica ya que explican que hace la función, y deben ser escritos de forma concisa y en tiempo presente.¹²

Las funciones permiten la modularización ya que cada función se encarga de una responsabilidad concreta. También permite devolver un resultado mediante el uso de return, lo que permite que el valor obtenido sea usado en otra parte del programa, una vez que un return es ejecutado, la función deja de ejecutarse.

Nuestro código tiene cada una de las opciones del menú como su propia función, además de tener funciones auxiliares como `validar_nombre()`.

Ordenamientos

Se utilizó `sorted()` para crear una nueva lista ordenada en la función `ordenar_paises()`. En nuestro proyecto se ordenan países por nombre, población ó superficie. En el caso de la superficie, se utilizó el keyword argument `reverse` en `sorted()` para así poder ordenar de forma ascendente o descendente.

El método `sorted()` toma dos argumentos opcionales, `key` y `reverse`.¹³

Estadísticas básicas

Las estadísticas permiten transformar los datos en información útil a ser analizada por el usuario, para esto no se utilizaron módulos. El proyecto fue creado para buscar el país con menor población, con mayor población, el promedio de población entre todos los países y el promedio de superficie entre todos los países, y por último la cantidad de países por continente. Se buscó utilizar un formateo del texto agradable para el usuario, agregando comas y eliminando los decimales más allá de los primeros dos.

Archivo CSV¹⁴

Un archivo CSV es un archivo de texto simple en el cual cada línea representa datos y estos datos están separados por comas, es comúnmente utilizado en hojas de cálculo debido a su bajo peso, simplicidad y facilidad de trabajo.

En un archivo CSV la primera fila actúa como la columna, siendo las siguientes filas los valores correspondientes a cada una de estas columnas.

Para trabajar con archivos CSV es necesario importar el módulo `csv`, al trabajar con estos archivos se utiliza un lector ó reader object junto con la función `csv.DictReader()` pasando como argumento de la función el archivo.

En nuestro programa utilizamos `DictReader()` porque esto interpreta la primera fila como nombres ó campos claves. Los países agregados se mantienen en memoria durante la ejecución del programa y se escriben en el archivo CSV cuando el usuario selecciona la opción de guardar cambios, garantizando así la permanencia de los datos.

Decisiones Técnicas y Arquitectura

Para el desarrollo de la aplicación se optó por una arquitectura modular basada en funciones, donde cada función posee una responsabilidad específica dentro del sistema. Esta decisión permitió mejorar la organización del código, facilitar su mantenimiento y evitar la duplicación de tareas.

La información de los países se almacena en memoria utilizando una lista de diccionarios. Cada diccionario representa un país y contiene los campos nombre, población, superficie y continente. Esta estructura fue elegida por su simplicidad y por permitir acceder fácilmente a cada dato mediante claves descriptivas.

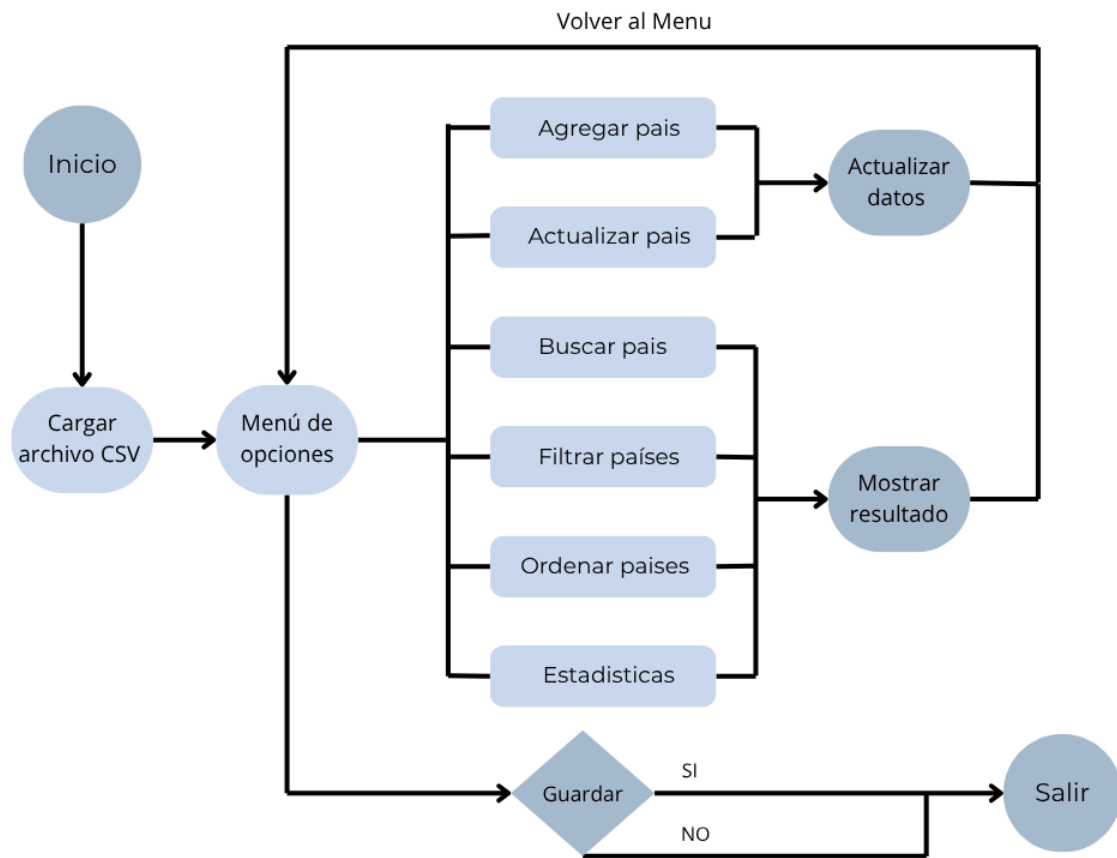
Los datos iniciales son cargados desde un archivo CSV utilizando la biblioteca estándar `csv` de Python. Durante la lectura se realizan validaciones para detectar errores de formato, registros incompletos o valores inválidos, evitando que información incorrecta afecte el funcionamiento del programa.

La función `menu()` muestra el menú y gestiona el flujo principal del programa. Desde ella además de mostrar las opciones disponibles para el usuario, delegando cada operación a funciones especializadas como agregar, actualizar, buscar, filtrar, ordenar o generar estadísticas. También se carga el archivo `países.csv` asegurándose de cargar un archivo que existe y tiene datos ingresados con el formato esperado.

Además, se implementaron funciones auxiliares reutilizables para tareas comunes, tales como la validación de nombres, la búsqueda de países por coincidencia parcial y la visualización de resultados. Esta estrategia permitió simplificar el código principal y mejorar su legibilidad.

Para los procesos de ordenamiento se utilizó la función incorporada `sorted()`, mientras que para los filtros y búsquedas se emplearon recorridos secuenciales sobre la lista de países. Las estadísticas se generan mediante acumuladores, contadores y comparaciones realizadas durante una única iteración de los datos.

El siguiente diagrama representa el flujo general de funcionamiento del sistema:



Ejemplos de Ejecución

A continuación se presentan algunas capturas de pantalla que muestran el funcionamiento de las principales funcionalidades implementadas en el programa.

Caso Opción 1 “Agregar país”:

```
ca: Símbolo del sistema - python buscador_de_paises.py

C:\Users\54117\Desktop\Nueva carpeta>python buscador_de_paises.py

=====
Ingrese el numero correspondiente a una de las siguientes opciones:
-----
1. Agregar pais
2. Actualizar pais
3. Buscar pais
4. Filtrar paises
5. Ordenar paises
6. Estadísticas
7. Guardar cambios y salir
=====

Ingrese una opcion: 1
Ingrese la opcion correspondiente:
      1 - Agregar pais
      2 - Volver al menu principal
      1
Ingrese el nombre del pais: UTN
Ingrese a que continente pertenece UTN: america del sur
Ingrese la poblacion de UTN: 2
Ingrese la superficie de UTN en kilometros cuadrados: 1
UTN,2,1,america del sur
Guardar los cambios? si/no: si
Ingrese la opcion correspondiente:
      1 - Agregar pais
      2 - Volver al menu principal
```

Caso opcion 6 “Estadísticas”:

```
=====
Ingrese una opcion: 6
El pais con la menor poblacion es St Pierre & Miquelon con 7026.
El pais con la mayor poblacion es China con 1313973713.
El promedio de poblacion entre todos los paises es 29,016,675.03 habitantes.
El promedio de superficie entre todos los paises es 603,932.49 kilometros cuadrados.
Cantidad de países por continente:
asia: 50
europa: 47
africa: 54
oceania: 20
america del norte: 36
america del sur: 13
=====
Ingrese el numero correspondiente a una de las siguientes opciones:
-----
1. Agregar pais
2. Actualizar pais
3. Buscar pais
4. Filtrar paises
5. Ordenar paises
6. Estadisticas
7. Guardar cambios y salir
=====
Ingrese una opcion: _
```

Espacio dejado intencionalmente en blanco

Caso opcion 4 “Filtrar paises”:

```
5. Ordenar paises
6. Estadisticas
7. Guardar cambios y salir
=====
Ingrese una opcion: 4
Ingrese la opcion correspondiente:
      1 - Filtrar paises por continente
      2 - Filtrar paises por poblacion
      3 - Filtrar paises por superficie
      4 - Volver al menu principal
      1
Ingrese el continente a buscar: asia

      Nombre: Afghanistan
      Poblacion: 31056997
      Superficie: 647500
      Continente: asia

      Nombre: Armenia
      Poblacion: 2976372
      Superficie: 29800
      Continente: asia

      Nombre: Azerbaijan
      Poblacion: 7961619
      Superficie: 86600
      Continente: asia
```

Espacio dejado intencionalmente en blanco

Referencias

1. [Documentacion oficial de python. Seccion 3.1.3. Lists](#)
2. [Think Python 2°ed, Capitulo 10, Seccion 10.1](#)
3. [Automate the boring stuff with Python, capitulo 6, lists, indexes](#)
4. [Documentacion oficial de python. Seccion 5.5. Dictionaries](#)
5. [Documentacion oficial de python. "Why must dictionary keys be immutable?"](#)
6. [Documentacion oficial de python. "How are dictionaries implemented in CPython?"](#)
7. [Documentacion oficial de Python. Seccion 4.7. Match statements](#)
8. [Documentacion oficial de Python. Seccion 4.1. if statements](#)
9. [Documentacion oficial de Python. Seccion 8.1. The if statement](#)
10. <https://greenteapress.com/thinkpython2/thinkpython2.pdf>
11. [Think Python 2°ed Capitulo 3. Seccion 3.7](#)
12. [Documentacion oficial de Python. Seccion 4.8. Defining functions](#)
13. [Documentacion oficial de Python. Sorted](#)
14. [Geeksforgeeks. Working with CSV files in Python.](#)

Dificultades y conclusiones

En cuanto al manejo de datos, hubo que normalizar el archivo CSV obtenido, este no tenía de valores adecuados para algunos campos, como por ejemplo continentes, y estos tuvieron que ser reemplazados, a su vez, al ingresar datos al archivo CSV surgieron dificultades por no tener datos normalizados, algo tan simple como un espacio en blanco generaba que el programa no se comporte como era esperado.

También estuvo la dificultad de tener valores iniciales para ciertas variables, como por ejemplo para la variable estadísticas, las variables minimo y maximo inicialmente eran inicializadas con un “número mágico” ya sea muy alto ó muy bajo, pero es confuso y mala práctica, por lo cual aprendimos a utilizar valores de el archivo csv para cumplir esa función, y estos valores serán reemplazados durante la ejecución del programa.

El proyecto fue muy util a la hora de practicar el manejo de archivos, lectura y escritura sobre estos, y sobre el manejo de listas de diccionarios, ademas de servir como practica general en el manejo de Python. A su vez fue una buena experiencia de trabajo de equipo debido a la division de tareas y charlas sobre como resolver problemas a la hora de como realizar el trabajo.

[Link](#) al repo:

https://github.com/GuidoToledo/buscador_de_paises.git

[Link](#) al Video de youtube:

https://youtu.be/WPZg3006-_E